

Example on Logistic Regression

H.V.Ha

08 - 06 - 2026

Contents

I. Introduction	1
II. Descriptive statistics	2
III. Logistic regression	6
IV. Plot the decision boundaries	10

I. Introduction

The objective of this example is to predict if a patient has diabetes for the Pima Indians Diabetes Data set using logistic regression.

Description of the dataset:

The data set is comprised of 768 observations and 9 variables. It is available in the package `mlbench`. We will be using `diabetes` as our response/target variable.

Data Description for the 9 variables are as follows.

- `pregnant`: number of times pregnant
- `glucose`: plasma glucose concentration (glucose tolerance test)
- `pressure`: diastolic blood pressure (mm Hg)
- `triceps`: triceps skin fold thickness (mm)
- `insulin`: 2-Hour serum insulin (μ U/ml)
- `mass`: Body Mass Index ($\text{weight in kg}/(\text{height in m})^2$)
- `pedigree`: diabetes pedigree function
- `age`: Age (years)
- `diabetes`: class variable (test for diabetes)

Install and load package:

```
#install.packages("mlbench")
library(mlbench)
```

Load the dataset:

```
data("PimaIndiansDiabetes")
head(PimaIndiansDiabetes) # show the first ten rows of the dataset
```

```
##   pregnant glucose pressure triceps insulin mass pedigree age diabetes
## 1         6     148       72      35        0 33.6    0.627  50      pos
## 2         1      85       66      29        0 26.6    0.351  31      neg
## 3         8     183       64       0        0 23.3    0.672  32      pos
## 4         1      89       66      23       94 28.1    0.167  21      neg
## 5         0     137       40      35      168 43.1    2.288  33      pos
## 6         5     116       74       0        0 25.6    0.201  30      neg
```

Display the structure of the dataset:

```
str(PimaIndiansDiabetes)

## 'data.frame': 768 obs. of 9 variables:
## $ pregnant: num 6 1 8 1 0 5 3 10 2 8 ...
## $ glucose : num 148 85 183 89 137 116 78 115 197 125 ...
## $ pressure: num 72 66 64 66 40 74 50 0 70 96 ...
## $ triceps : num 35 29 0 23 35 0 32 0 45 0 ...
## $ insulin : num 0 0 0 94 168 0 88 0 543 0 ...
## $ mass : num 33.6 26.6 23.3 28.1 43.1 25.6 31 35.3 30.5 0 ...
## $ pedigree: num 0.627 0.351 0.672 0.167 2.288 ...
## $ age : num 50 31 32 21 33 30 26 29 53 54 ...
## $ diabetes: Factor w/ 2 levels "neg","pos": 2 1 2 1 2 1 2 1 2 2 ...
```

II. Descriptive statistics

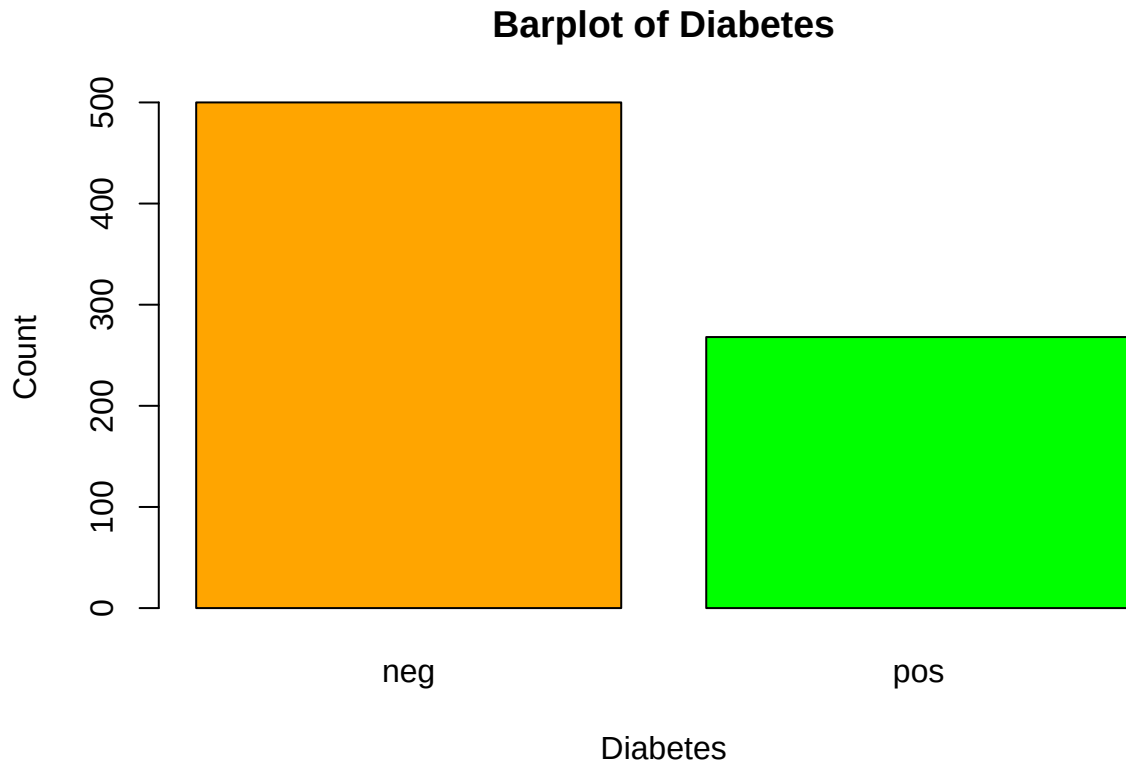
First, we make some descriptive statistics in order to summarize characteristics of the dataset.

```
summary(PimaIndiansDiabetes)
```

	pregnant	glucose	pressure	triceps	
## Min. :	0.000	Min. : 0.0	Min. : 0.00	Min. : 0.00	
## 1st Qu.:	1.000	1st Qu.: 99.0	1st Qu.: 62.00	1st Qu.: 0.00	
## Median :	3.000	Median :117.0	Median : 72.00	Median :23.00	
## Mean :	3.845	Mean :120.9	Mean : 69.11	Mean :20.54	
## 3rd Qu.:	6.000	3rd Qu.:140.2	3rd Qu.: 80.00	3rd Qu.:32.00	
## Max. :	17.000	Max. :199.0	Max. :122.00	Max. :99.00	
	insulin	mass	pedigree	age	diabetes
## Min. :	0.0	Min. : 0.00	Min. :0.0780	Min. :21.00	neg:500
## 1st Qu.:	0.0	1st Qu.:27.30	1st Qu.:0.2437	1st Qu.:24.00	pos:268
## Median :	30.5	Median :32.00	Median :0.3725	Median :29.00	
## Mean :	79.8	Mean :31.99	Mean :0.4719	Mean :33.24	
## 3rd Qu.:	127.2	3rd Qu.:36.60	3rd Qu.:0.6262	3rd Qu.:41.00	
## Max. :	846.0	Max. :67.10	Max. :2.4200	Max. :81.00	

Plot a barplot for the variables diabetes:

```
diabetes_counts <- table(PimaIndiansDiabetes$diabetes)
# Create barplot of diabetes variable
barplot(diabetes_counts, xlab = "Diabetes", ylab = "Count",
        col = c("orange", "green"),
        main = "Barplot of Diabetes")
```



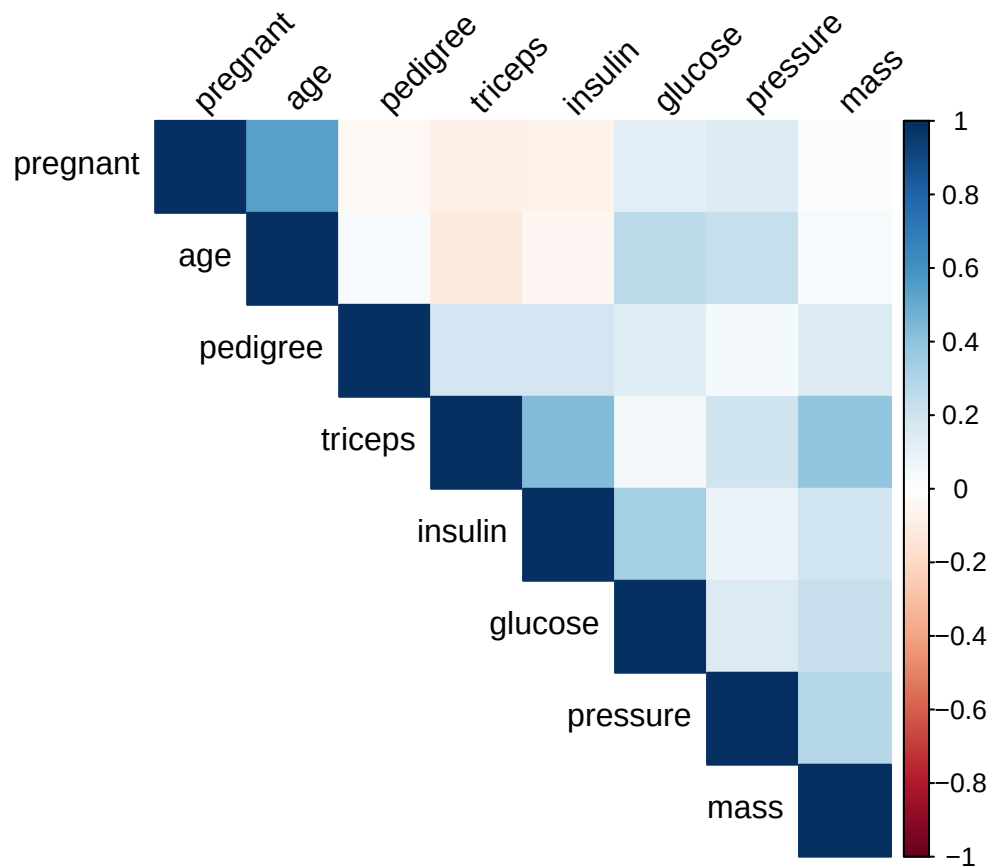
Compute correlation matrix for the numerical variables (except “diabetes”):

```
corr_matrix <- cor(PimaIndiansDiabetes[, -9]) # Remove the categorical variable diabetes
print(round(corr_matrix, 4))
```

```
##      pregnant glucose pressure triceps insulin mass pedigree age
## pregnant  1.0000  0.1295   0.1413 -0.0817 -0.0735 0.0177 -0.0335 0.5443
## glucose   0.1295  1.0000   0.1526  0.0573  0.3314 0.2211  0.1373 0.2635
## pressure  0.1413  0.1526   1.0000  0.2074  0.0889 0.2818  0.0413 0.2395
## triceps   -0.0817 0.0573   0.2074  1.0000  0.4368 0.3926  0.1839 -0.1140
## insulin   -0.0735 0.3314   0.0889  0.4368  1.0000 0.1979  0.1851 -0.0422
## mass       0.0177 0.2211   0.2818  0.3926  0.1979 1.0000  0.1406 0.0362
## pedigree  -0.0335 0.1373   0.0413  0.1839  0.1851 0.1406  1.0000 0.0336
## age        0.5443 0.2635   0.2395 -0.1140 -0.0422 0.0362  0.0336 1.0000
```

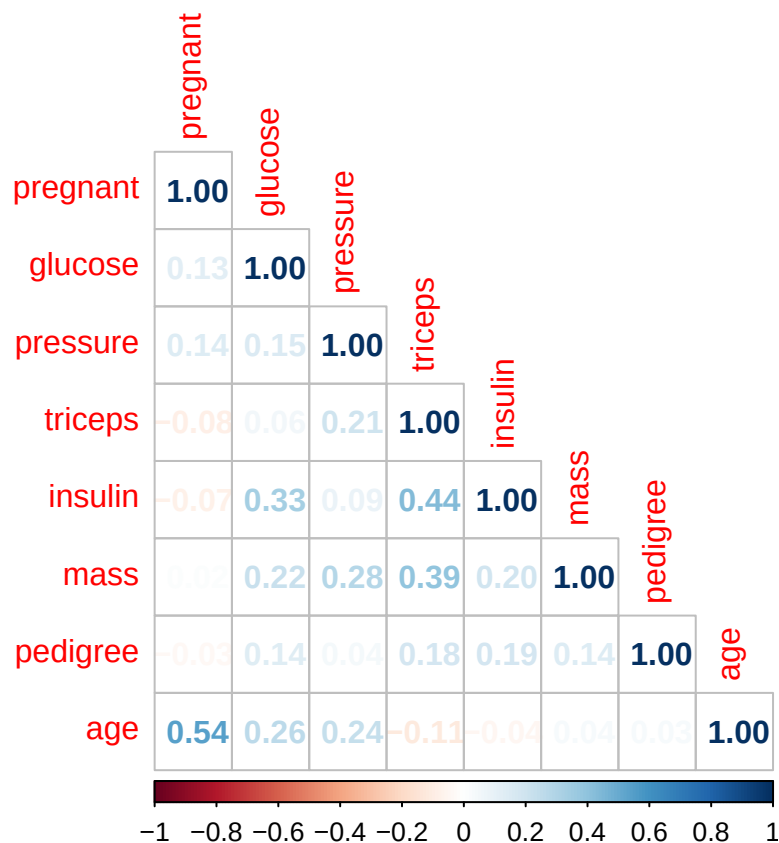
Plot a correlation matrix graph (using corrplot package, check if you already have the corrplot package or install it):

```
library(corrplot)
# Plot correlation matrix
corrplot(corr_matrix, method = "color", type = "upper", order = "hclust",
          tl.col = "black", tl.srt = 45)
```



Or a correlation plot with number

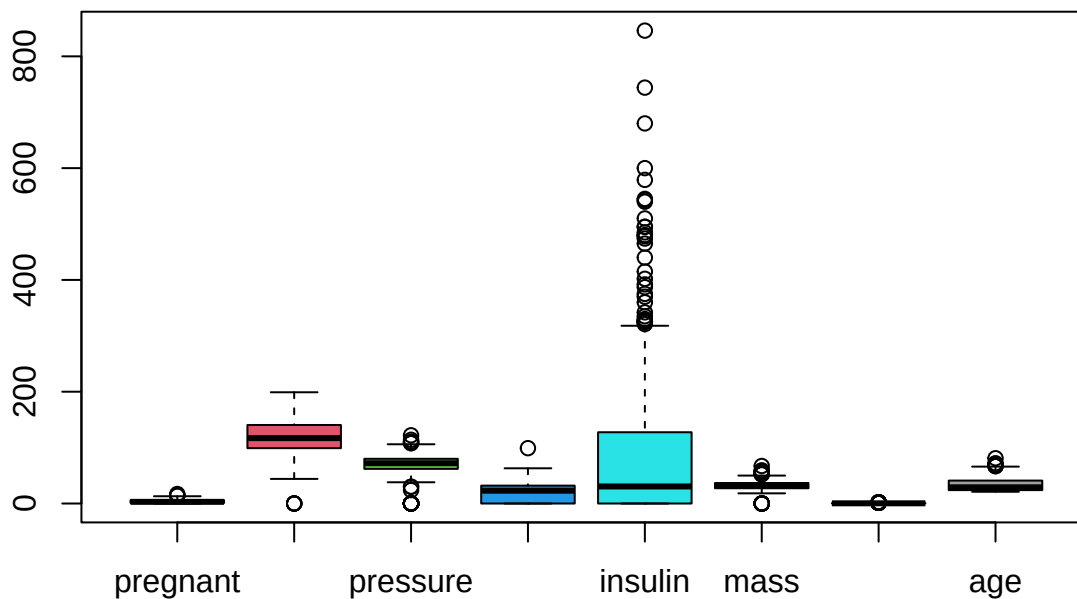
```
corrplot(corr_matrix, type = "lower", method = "number")
```



Plot boxplots for numerical variables:

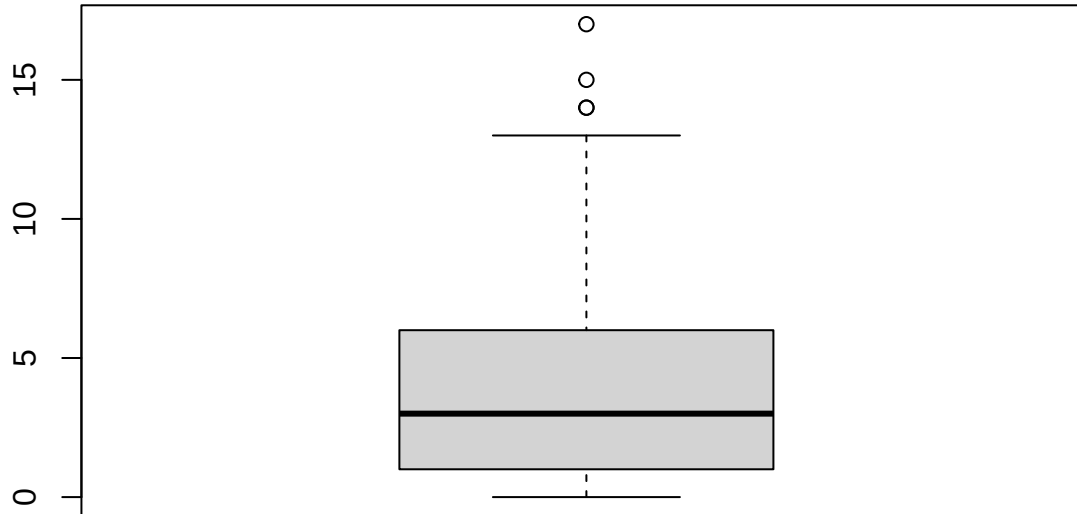
```
# Remove diabetes column from data frame
diabetes_removed <- PimaIndiansDiabetes[, -9]

boxplot(diabetes_removed, col = c(1:8))
```



We can see that the variables don't have the same scale. So you can try to plot the boxplot of each variable separately. For example, plot a boxplot for the 'pregnant' variable:

```
boxplot(diabetes_removed[, 'pregnant'])
```



III. Logistic regression

III.1. Split dataset into train and test sets

```
set.seed(123)
train_index <- sample(nrow(PimaIndiansDiabetes),
                      size = 0.7*nrow(PimaIndiansDiabetes),
                      replace = FALSE)

train_data <- PimaIndiansDiabetes[train_index, ]
test_data <- PimaIndiansDiabetes[-train_index, ]
```

Check the dimension of the training dataset and testing dataset

```
dim(train_data)
```

```
## [1] 537  9
```

```
dim(test_data)
```

```
## [1] 231  9
```

III.2. Fit the model with all dependent variables

We have 8 independent variables: “pregnant”, “glucose”, “pressure”, “triceps”, “insulin”, “mass”, “pedigree”, “age”, “diabetes”.

```
fit_model <- glm(diabetes ~ ., data = train_data, family = "binomial")
# diabetes ~ . is equivalent to
# diabetes ~ pregnant + glucose + pressure + triceps + ... + age
# that is, if you want to fit the model with all variables in the dataset,
# just use diabetes ~ .
summary(fit_model)
```

```
##
```

```
## Call:
```

```
## glm(formula = diabetes ~ ., family = "binomial", data = train_data)
```

```
##
## Coefficients:
##           Estimate Std. Error z value Pr(>|z|)
## (Intercept) -8.405409   0.841872  -9.984  < 2e-16 ***
## pregnant    0.103471   0.037973   2.725  0.00643 **
## glucose     0.035730   0.004563   7.830  4.89e-15 ***
## pressure    -0.012707   0.006057  -2.098  0.03590 *
## triceps     0.003563   0.008088   0.440  0.65959
## insulin     -0.001710   0.001060  -1.613  0.10671
## mass        0.088735   0.017954   4.942  7.72e-07 ***
## pedigree    0.696250   0.334761   2.080  0.03754 *
## age         0.017015   0.011066   1.538  0.12415
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 694.17  on 536  degrees of freedom
## Residual deviance: 509.76  on 528  degrees of freedom
## AIC: 527.76
##
## Number of Fisher Scoring iterations: 5
```

We can see that the intercept has a coefficient of -8.404762, which indicates the log-odds of the outcome for the reference group (the group with all predictor variables set to 0). The other predictor variables have positive or negative coefficients, which indicate the direction and magnitude of their effect on the log-odds of the outcome, holding all other variables constant. The p-values indicate whether each variable is statistically significant in the model.

We remove triceps, insulin and age since these variables are not significant, and run again the model:

```
fit_model2 <- glm(diabetes ~ pregnant + glucose + pressure + mass +
                  pedigree, data = train_data, family = "binomial")
summary(fit_model2)
```

```
##
## Call:
## glm(formula = diabetes ~ pregnant + glucose + pressure + mass +
##      pedigree, family = "binomial", data = train_data)
##
## Coefficients:
##           Estimate Std. Error z value Pr(>|z|)
## (Intercept) -7.799784   0.775631 -10.056  < 2e-16 ***
## pregnant    0.138138   0.032715   4.223  2.42e-05 ***
## glucose     0.034314   0.004101   8.367  < 2e-16 ***
## pressure    -0.011448   0.005844  -1.959  0.0501 .
## mass        0.084610   0.016826   5.028  4.94e-07 ***
## pedigree    0.676771   0.330840   2.046  0.0408 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 694.17  on 536  degrees of freedom
## Residual deviance: 515.01  on 531  degrees of freedom
## AIC: 527.01
```

```
##
## Number of Fisher Scoring iterations: 5
```

To interpret the odds with respect to each variable, you can exponentiate the coefficient of each variable to obtain the odds ratio. An odds ratio greater than 1 indicates that the odds of the outcome variable (diabetes) increase with a unit increase in the predictor variable, while an odds ratio less than 1 indicates that the odds decrease with a unit increase in the predictor variable.

```
exp(coef(fit_model2))
```

```
## (Intercept)    pregnant    glucose    pressure    mass    pedigree
## 0.0004098236 1.1481340671 1.0349095505 0.9886175150 1.0882925528 1.9675146352
```

For instance, the coefficient for glucose is 0.034314, indicating that for a one-unit increase in glucose, the log odds of having diabetes increases by 0.034314. To obtain the odds ratio, we exponentiate the coefficient: $\exp(0.034314) = 1.03491$. That is, the odds ratio for glucose is 1.03491, which means that for a one-unit increase in glucose, the odds of having diabetes increase by 3.49%.

Furthermore, the coefficient of pressure is -0.011448, which means that for every one-unit increase in pressure, the log-odds of having diabetes decreases by 0.011448. To convert this to odds, you can take the exponent of the coefficient: $\exp(-0.011448) = 0.9886$. This means that for every one-unit increase in pressure, the odds of having diabetes decrease by a factor of 0.9886, or equivalently, the odds decrease by about $100\% - 98.86\% = 1.14\%$ for every one-unit increase in pressure.

You can make similar interpretation for the other variables.

III.3. Prediction on the training dataset

Compute the predicted probabilities:

```
PredictTrain <- predict(fit_model2, type = "response")
```

Create variable 'predictTrain_diabetes' with values in {pos, neg}. We choose the threshold is 0.5, that is:

- If $P(Y = 1) > 0.5$, predict 1
- If $P(Y = 1) < 0.5$, predict 0

```
predictTrain_diabetes <- ifelse(PredictTrain > 0.5, "pos", "neg")
```

```
confusion_matrix_train <- table(predictTrain_diabetes, train_data$diabetes)
colnames(confusion_matrix_train) <- c("True neg Diabetes", "True pos diabetes")
rownames(confusion_matrix_train) <- c("Pred neg Diabetes", "Pred pos diabetes")
confusion_matrix_train
```

Compute confusion matrix

```
##
## predictTrain_diabetes True neg Diabetes True pos diabetes
##    Pred neg Diabetes      308      82
##    Pred pos diabetes      42      105
```

```
Compute misclassification rate, false positive rate and false negative rate false_positive_rate <-
confusion_matrix[2,1]/sum(confusion_matrix[,1]) false_positive_rate
false_negative_rate <- confusion_matrix[1,2]/sum(confusion_matrix[,2]) false_negative_rate
```

```

misclassification_rate <- mean(predictTrain_diabetes != train_data$diabetes)
false_positive_rate <- confusion_matrix_train[2,1]/sum(confusion_matrix_train[,1])
false_negative_rate <- confusion_matrix_train[1,2]/sum(confusion_matrix_train[,2])
misclassification_rate <- round(misclassification_rate, 4)
false_positive_rate <- round(false_positive_rate, 4)
false_negative_rate <- round(false_negative_rate, 4)

```

```

sprintf("Mis-classification error is %s",misclassification_rate)

```

Compute the misclassification rate

```

## [1] "Mis-classification error is 0.2309"

```

```

accuracy_train = 1 - misclassification_rate

```

```

sprintf("Accuracy is %s",accuracy_train)

```

```

## [1] "Accuracy is 0.7691"

```

```

sprintf("False positive rate is: %s", false_positive_rate)

```

```

## [1] "False positive rate is: 0.12"

```

```

sprintf("False negative rate is: %s", false_negative_rate)

```

```

## [1] "False negative rate is: 0.4385"

```

III.4. Prediction on the test dataset

```

PredictTest <- predict(fit_model2, type = "response", newdata = test_data)
predictTest_diabetes <- ifelse(PredictTest > 0.5, "pos", "neg")
confusion_matrix_test <- table(predictTest_diabetes, test_data$diabetes)
colnames(confusion_matrix_test)<-c("True neg Diabetes",'True pos diabetes')
rownames(confusion_matrix_test)<-c("Pred neg Diabetes",'Pred pos diabetes')
confusion_matrix_test

```

```

##

```

```

## predictTest_diabetes True neg Diabetes True pos diabetes
##   Pred neg Diabetes      133          35
##   Pred pos diabetes      17          46

```

```

misclassification_rate_test <- mean(predictTest_diabetes != test_data$diabetes)
false_positive_rate_test <- confusion_matrix_test[2,1]/sum(confusion_matrix_test[,1])
false_negative_rate_test <- confusion_matrix_test[1,2]/sum(confusion_matrix_test[,2])
misclassification_rate_test <- round(misclassification_rate_test, 4)
false_positive_rate_test <- round(false_positive_rate, 4)
false_negative_rate_test <- round(false_negative_rate_test, 4)

```

```

sprintf("Mis-classification error on test set is %s",misclassification_rate_test)

```

Compute misclassification rate on test dataset

```

## [1] "Mis-classification error on test set is 0.2251"

```

```

accuracy_test = 1 - misclassification_rate_test

sprintf("Accuracy is %s",accuracy_test)

## [1] "Accuracy is 0.7749"

sprintf("False positive rate on test set is: %s", false_positve_rate_test)

## [1] "False positive rate on test set is: 0.12"

sprintf("False negative rate on test set is: %s", false_negative_rate_test)

## [1] "False negative rate on test set is: 0.4321"

```

IV. Plot the decision boundaries

From the results, we can observe that the two strongest predictors of the outcome are glucose rate and BMI (mass). High glucose rate and high BMI are strong indicators of Diabetes. We will plot the boundaries using training dataset.

```

library(ggplot2)
# Get the predicted values on the train dataset
train_data$Predicted <- fit_model2$fitted.values

# Plot the two variales: glucose and mass (BMI) with respect to Predicted variable
ggplot(train_data, aes(x = mass, y = glucose, color = Predicted > 0.5)) +
  geom_point(size=2, alpha=0.5)

```

